

고급 과제 - 2

2024405020 로봇학부 주호진

제출 목록

1. LLM 입력 프롬프트

도서 대출 관리 시스템을 python(.py) 코드로 짜줘

조건은 다음과 같아

책제목과 사용자에는 공백 없음, 수량은 0 이상의 정수(ex. 1, 2, 3, 4...), 정해진 명령어 외에 잘못된 명령어 형식은 ERROR로 예외처리할 것, 함수(function)로 기능 분리할 것, eval() 절대 사용 금지

while True문 안에 input으로 사용자에게 아래의 명령어를 Exit 전까지 계속 입력받도록 해주고 오류없는지 교차검증 여러번 진행 후 답변해

입력값: add 책제목 수량 (ex. add 강아지통 3)

- 책제목과 수량 등록, 이미 있는 책제목일 경우 수량 추가

입력값: borrow 사용자 책제목 (ex. borrow 주호진 강아지통)

- 책이 있고 수량이 남아 있으며, 해당 사용자가 동일한 책을 대출 중이 아닌 경우에만 대출 가능

- 실패 시 ERROR 출력 + 사유 출력(ex. 남아있는 책 재고가 없습니다, 동일한 책을 대출 중입니다, 해당 도서관에 존재하지 않는 도서입니다 등)

입력값: return 사용자 책제목 (ex. return 주호진 강아지통)

- 해당 사용자가 대출 중인 책만 반납 가능

- 실패 시 ERROR 출력 + 사유 출력(대출 중이 아닌 책입니다, 해당 도서관에 존재하지 않는 도서입니다 등)

입력값: status 책제목 (ex. status 강아지통)

- 책의 남은 수량 출력

- 책이 존재하지 않으면 ERROR 출력

입력값: user 사용자 (ex. user 주호진)

- 해당 사용자의 대출 목록 전부 출력

- 대출 목록이 없으면 EMPTY 출력

입력값: list

- 전체 책 목록을 등록 순서대로 출력

입력값: exit

- 종료(break 사용)

도서 대출 관리 시스템을 python(.py) 코드로 짜줘

조건은 다음과 같아

책제목과 사용자에는 공백 없음, 수량은 0 이상의 정수(ex. 1, 2, 3, 4...), 정해진 명령어 외에 잘못된 명령어 형식은 ERROR로 예외처리할 것, 함수(function)로 기능 분리할 것, eval() 절대 사용 금지

while True문 안에 input으로 사용자에게 아래의 명령어를 Exit 전까지 계속 입력받도록 해주고 오류없는지 교차검증 여러번 진행 후 답변해

입력값: add 책제목 수량 (ex. add 강아지똥 3)

- 책제목과 수량 등록, 이미 있는 책제목일 경우 수량 추가

입력값: borrow 사용자 책제목 (ex. borrow 주호진 강아지똥)

- 책이 있고 수량이 남아 있으며, 해당 사용자가 동일한 책을 대출 중이 아닌 경우에만 대출 가능
- 실패 시 ERROR 출력 + 사유 출력(ex. 남아있는 책 재고가 없습니다, 동일한 책을 대출 중입니다, 해당 도서관에 존재하지 않는 도서입니다 등)

입력값: return 사용자 책제목 (ex. return 주호진 강아지똥)

- 해당 사용자가 대출 중인 책만 반납 가능
- 실패 시 ERROR 출력 + 사유 출력(대출 중이 아닌 책입니다, 해당 도서관에 존재하지 않는 도서입니다 등)

입력값: status 책제목 (ex. status 강아지똥)

- 책의 남은 수량 출력
- 책이 존재하지 않으면 ERROR 출력

입력값: user 사용자 (ex. user 주호진)

- 해당 사용자의 대출 목록 전부 출력
- 대출 목록이 없으면 EMPTY 출력

입력값: list

- 전체 책 목록을 등록 순서대로 출력

입력값: exit

- 종료(break 사용)

2. 프롬프트 입력 후 LLM 첫 출력 코드 (.py 형태)

https://github.com/j00h0jin/robit_python/blob/main/0713_h2_reference.py

```
📁 도서 대출 관리 시스템 코드 ( library_system.py )

Python

class LibrarySystem:
    def __init__(self):
        # 도서 목록을 저장하는 딕셔너리 (책제목: 남은수량)
        # Python 3.7+부터 딕셔너리는 삽입 순서를 유지하므로 list 연결어 세 등록 순서대로 출력
        self.books = {}
        # 사용자별 대출 목록을 저장하는 딕셔너리 (사용자: [대출한 책제목 리스트])
        self.users = {}

    def add_book(self, title, quantity_str):
        # 수량이 0 이상의 정수인지 확인 (isdigit())은 음수나 소수를 False로 처리)
        if not quantity_str.isdigit():
            print("ERROR: 수량은 0 이상의 정수여야 합니다.")
            return

        quantity = int(quantity_str)
        if title in self.books:
            self.books[title] += quantity
        else:
            self.books[title] = quantity
        print(f"[성공] '{title}' 도서가 {quantity}권 추가되었습니다. (현재 스톡 {self.books[title]}개)")

    def borrow_book(self, user, title):
        if title not in self.books:
            print("ERROR: 해당 도서관에 존재하지 않는 도서입니다.")
            return

        # 사용자가 대출 기록이 없으면 빈 리스트 생성
        if user not in self.users:
            self.users[user] = []

        if title in self.users[user]:
            print("ERROR: 동일한 책을 대출 중입니다.")
            return

        if self.books[title] <= 0:
            print("ERROR: 남아있는 책 재고가 없습니다.")
            return

        # 대출 처리
        self.books[title] -= 1
        self.users[user].append(title)
        print(f"[성공] {user}님이 '{title}' 도서를 대출하셨습니다.")

    def return_book(self, user, title):
        if title not in self.books:
            print("ERROR: 해당 도서관에 존재하지 않는 도서입니다.")
            return
```

+ Gemini에게 물어보기

Pro



3. 오류 및 개선점 분석

생각보다 예외처리가 잘 되어 있어서 큰 오류를 발견하지 못했다. 주석도 달아주어서 LLM이 단 주석과 구분하기 위해 `''' '''` 주석을 이용하면서 코드리뷰를 진행하였다.

```
# 사용자가 대출 기록이 없으면 빈 리스트 생성
if user not in self.users:
    self.users[user] = []
```

```
if user not in self.users or title not in self.users[user]:
    print("ERROR: 대출 중이 아닌 책입니다.")
    return
```

```
def user_info(self, user):
    if user not in self.users or not self.users[user]:
        print("EMPTY")
    return
```

크게 문제되는 부분은 아니지만 borrow를 할 때 user 정보를 생성하는 코드가 있기 때문에 해당 부분을 경우에 따라 유저 정보가 아예 없는 경우, 정보는 있지만 책 목록이 없는 경우를 분리하여 ERROR 메시지를 아래와 같이 구분해주었다.

```
if user not in self.users:
    print("ERROR: 등록되지 않은 사용자입니다.")
    return

if title not in self.users[user]:
    print("ERROR: 대출 중이 아닌 책입니다.")
    return
```

```
def user_info(self, user):
    if user not in self.users:
        print("EMPTY: 등록되지 않은 사용자입니다.")
        return
    if not self.users[user]:
        print("EMPTY: 대출 목록이 없습니다.")
    return
```

`++isdigit()`은 단일 글자가 숫자 모양으로 생겼으면 True 반환하여서 2^3 , 3 , $_3$, $③$ 과 같은 것도 True로 반환한다고 한다. `Isdecimal()`은 int형 변환이 가능한지 확인해주므로 `isdecimal()`로 변경해주었다.

4. 개선한 코드(.py)

https://github.com/j00h0jin/robit_python/blob/main/0713_h2_reference.py

5. LLM 프롬프트 입력 단계부터 오류 개선까지의 고찰

LLM으로 코드를 짜는 것은 내가 직접 짜는 것이 아니기 때문에 최대한 세세한 부분까지 프롬프트에 입력해서 내가 원하는 의도에 최대한 적합한 코드가 나오도록 했다. 입력 단계에서 예시나 중의적으로 해석될 여지가 있는 어휘들을 수정한 후 입력하였으며, 예시 등을 추가하여 의도한 바에 가까운 코드가 출력되도록 유도하였다.

오류 개선은 일단 치명적인 오류가 있는지 확인하기 위해 LLM에게 테스트 코드를 달라고 하여 테스트해보았다. 그리고 예외처리한 부분 외에 다른 허점이 있는지를 확인하기 위해 다양한 케이스들을 실험해본다. 작동 상으로 문제가 없을 경우 코드를 자세히 리뷰하며 구조나 흐름상 부자연스러운 부분이 있는지 확인하여 수정하였다.